



P11

Développement d'un Caterpillar



1 Consignes

1.1 Contexte

L'objectif de ce mini-projet est la création du fameux jeu Snake ! Ce jeu formidable développé en 1976 pour des bornes d'arcades a vu de nombreuses versions se succéder. Vous êtes aujourd'hui confrontés à un paradoxe : créer un Snake sans utiliser Python ni Anaconda ! Eh oui même si ce jeu est incontournable, vos employeurs sont ophiophobes ! De fait, ils vous demandent de développer en C# une version de ce jeu, Caterpillar, où une chenille veut devenir un papillon. Pour cela, la chenille va manger tout ce qui se trouve sur son passage.

1.2 Le jeu

Le principe du jeu est très simple :

- La chenille a pour objectif de manger les feuilles tombant des arbres aléatoirement sur le plateau de jeu
- Lorsque la chenille mange une feuille elle grossit d'un segment
- Attention la chenille possède des glandes odoriférantes comme les punaises. La différence ? Elle dégage une odeur de chocolat qui attirera immédiatement les oiseaux qui la mangeront. Pour ne pas activer ces glandes, la chenille doit éviter de boucler sur elle même.
- Lorsque la chenille atteint trois fois la taille de son espace vital, elle peut alors entamer son processus de métamorphose pour devenir un papillon.

L'objectif du joueur, aussi accro au chocolat soit-il, est d'aider la chenille à devenir un papillon.

1.3 Le développement

Afin de faciliter le développement de ce jeu qui promet d'être révolutionnaire, il est demandé de matérialiser les différents éléments comme suit :

- L'espace vital de la chenille (*gameBoard*) est matérialisé par un tableau 2D, dont la taille est choisie par le joueur
- La chenille est matérialisée par deux tableaux 1D. Le premier (*playerRow*) correspond à la position de chaque segment de son corps sur les lignes du tableau 2D. Le second (*playerCol*) correspond à la position de chaque segment sur les colonnes du tableau.
- Chaque feuille sera matérialisée par son emplacement en ligne (*tokenRow*) et en colonne (*tokenCol*).

1.4 Merci les stagiaires ! (peut-être)

Puisque l'entreprise qui vous demande de développer Caterpillar est désormais très réputée, une armée de stagiaires est engagée chaque année et il se trouve que deux stagiaires ont été en concurrence pour commencer le développement du jeu. Ainsi, vous avez le choix de reprendre le squelette de l'un ou l'autre des stagiaires.

Le premier stagiaire (`caterpillar_squelet1.cs`) a eu beaucoup de mal et n'a laissé qu'un squelette globalement vide, aussi plus de travail sera requis pour reprendre la suite de ce stagiaire.

Le second stagiaire (`caterpillar_squelet2.cs`) a été en mesure d'avancer un peu plus, en particulier certaines fonctionnalités plus compliquées à implémenter sont déjà codées mais doivent être expliquées à votre futur employeur car aucun commentaire n'est présent sur ces fonctionnalités.

Par la suite, chaque étape requise pour chacun des stagiaires est distinguée par une $\star$ pour le Stagiaire 1 et une $\dagger$ pour le Stagiaire 2. Si aucun symbole n'est présent, dans ce cas l'étape doit être implémentée pour les deux stagiaires.

1.5 Dernières consignes

Les squelettes proposés actuellement sont nécessairement incomplets, soyez libre de rajouter des variables et constantes ou encore des fonctions et procédures dont vous avez besoin.

Il est important de noter que vos employeurs veulent comprendre votre développement, commentaires et conventions de nommage sont donc à respecter. De plus, tout usage de fonctionnalités autres que celles vues en cours de P11 sont interdites (eh oui l'employeur est parmi vous !).

Enfin, il n'est pas nécessaire de suivre l'ordre linéaire décrit ci-après pour développer correctement votre jeu. Il vous est même vivement conseillé d'implémenter les fonctions manquantes dès qu'elles sont mentionnées dans la Section 2.

2 Mise en place du jeu

2.1 Initialisation du jeu

Avant de commencer le jeu, plusieurs initialisations sont à implémenter :

- La taille de l'espace vital (*gameSize*) de la chenille doit être demandée au joueur
- L'espace vital est créé sous la forme d'un tableau 2D de taille *gameSize* $\times$ *gameSize*. Ce tableau est initialisé automatiquement avec des ' Δ '.
- Les tableaux de position du joueur en ligne (*playerRow*) et en colonne (*playerCol*) doivent être initialisés à -1 et de taille $3 \times \text{gameSize}$. La tête de la chenille est toujours à la position 0 des tableaux. Ainsi les coordonnées de la tête dans le tableau sont : (*playerRow*[0], *playerCol*[0]). La tête de la chenille est placée par défaut en bas à gauche de son espace vital.
- La position de la feuille (*tokenRow* et *tokenCol*) doit être initialisée aléatoirement.
- Mettre à jour l'espace vital de la chenille à l'aide de la procédure *updateTable()* (voir Section 3).
- Afficher l'espace de jeu dans la console à l'aide de la procédure *displayBoard()* (voir Section 4).

2.2 Déroulement du jeu

Ici l'idée est de mettre en place la logique de jeu, à savoir qu'une phase de jeu se déroule en plusieurs étapes:

1. Le joueur appuie sur une touche pour indiquer la direction dans laquelle doit se déplacer la chenille. Si la touche demandée n'est pas valide, l'algorithme doit demander un nouveau mouvement sans actualiser le jeu. Les touches autorisées sont les suivantes :

- 'z' → déplacement vers le haut
 - 'q' → déplacement vers la gauche
 - 's' → déplacement vers le bas
 - 'd' → déplacement vers la droite
 - 'm' → on arrête le jeu
2. Traduire le mouvement en déplacement sur les lignes ou sur les colonnes à l'aide des variables *incR* et *incC*.
 3. Vérifier que si on applique le déplacement à la tête elle ne touche pas le corps de la chenille à l'aide de la fonction *checkPerdu()* (voir Section 5).
 4. Vérifier que si on applique le déplacement, la chenille ne sort pas de son espace vital. Si elle sort de son espace vital, le jeu reste inchangé et le joueur n'a pas perdu.
 5. Déplacer le joueur si toutes les vérifications sont satisfaites à l'aide de la procédure *movePlayer()* (voir Section 6).
 6. Créer une nouvelle feuille si la chenille a mangé la précédente.
 7. Mettre à jour l'affichage du jeu à l'aide de la procédure *updateTable()* (voir Section 3).
 8. Afficher le nouvel état du jeu à l'aide de la procédure *displayBoard()* (voir Section 4).
 9. La logique de jeu continue jusqu'à ce que :
 - le joueur ait gagné → sa taille est égale à 3 fois la taille de l'espace vital
 - le joueur ait perdu → la chenille a rencontré son corps et a libéré une odeur de chocolat qui attire les oiseaux
 - le joueur a décidé de quitter le jeu → la touche 'm' a été pressée.

★ **Stagiaire 1 :**

Ne vous inquiétez pas ça va bien se passer !

† **Stagiaire 2 :**

Le stagiaire a réussi à faire les étapes 2 et 4, mais il serait bien de vérifier qu'une erreur ne s'y trouve pas.
Proposer différents tests qui permettent d'assurer que le stagiaire ne s'est pas trompé, et corriger le code au besoin.

2.3 Fin de jeu

Une fois la logique de jeu terminée, il ne reste plus qu'à afficher un message de victoire ou de défaite au joueur en fonction de sa performance dans le jeu.

3 Mise à jour de l'espace de jeu

L'espace de jeu va être modifié au cours du temps, il est donc nécessaire de renouveler son affichage en mettant à jour les informations du jeu au fur et à mesure que le joueur déplace la chenille. Pour cela, le tableau 2D *gameBoard* fait office de carte du jeu.

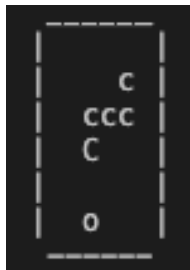
L'objectif ici est de développer une procédure permettant de mettre à jour le jeu. Celle-ci doit :

- Placer un '△' si l'espace est vide.
- Placer un 'o' à l'emplacement de la feuille.
- Placer un 'C' pour la tête de la chenille.
- Placer un 'c' pour chaque segment de son corps.

4 Affichage de l'espace de jeu

L'objectif ici est de développer une procédure qui permettra d'afficher le jeu sur la console. Pensez à bien nettoyer la console à l'aide de la fonction *Console.Clear()*.

Il est important que le cadre du jeu apparaisse à l'écran comme suit (ici le joueur a choisi une taille de 6 et a déjà récupéré 4 feuilles) :



★ Stagiaire 1 :

Pas de chance, le stagiaire n'était pas encore arrivé à ce stade !

† Stagiaire 2 :

Quelle chance ! Le stagiaire a laissé quelques notes :

- Commencer par afficher la bordure du haut
- Afficher un '|' au début et à la fin de chaque ligne du tableau
- Finir par afficher la bordure du bas

5 Game Over

L'objectif ici est de vérifier que le joueur n'a pas perdu. Pour cela, il faut vérifier que le mouvement demandé par l'utilisateur va faire boucler la chenille : la tête se trouve sur les mêmes coordonnées que l'un des segments de son corps.

Attention il n'est pas demandé ici de bouger la chenille, seulement de vérifier que si l'on bouge la tête, celle-ci rencontrera son corps.

6 Déplacement du joueur

L'objectif de cette partie est de faire se déplacer la chenille en fonction de ce qu'a demandé l'utilisateur. Sur ce point les deux stagiaires ont mené des travaux bien différents.

★ Stagiaire 1 :

Le stagiaire fidèle à lui même n'a pas réussi à implémenter cette fonction non plus, mais malgré tout il a laissé quelques notes sur le déplacement de la chenille.

Deux cas sont à distinguer :

1. Le cas où la chenille a attrapé la feuille : ici la chenille va gagner un nouveau segment, donc il faut conserver toutes les anciennes positions du corps de la chenille et insérer une nouvelle position, celle de la feuille, au début du tableau. (voir exemple ci dessous)
2. Le cas où la chenille n'a pas attrapé la feuille : ici il faut insérer la nouvelle position au début et propager la position des segments de la chenille. À savoir, le segment 3 prend la position du segment 2, le segment 4 prend la position du segment 3, etc... afin que la chenille avance. De cette façon la position du dernier segment sera oubliée. (voir exemple ci dessous)

Prenons un exemple avec $playerRow = \{0,1,-1\}$ et $playerCol = \{1,1,-1\}$

Si la feuille est en position (0,0) et que la chenille bouge à gauche : $playerRow = \{0,0,1\}$ et $playerCol = \{0,1,1\}$

Si à la place la chenille bouge en haut : $playerRow = \{0,0,-1\}$ et $playerCol = \{2,1,-1\}$

Ces exemples sont illustrés par des figures disponibles dans les fichiers `Cas1.png` et `Cas2.png`.

† Stagiaire 2 :

Ici le stagiaire a déjà tout codé mais n'a laissé aucun commentaire et son code n'est pas vraiment compréhensible.

Il vous est demandé ici de vérifier le code produit par le stagiaire et de le commenter. Pour justifier de vos commentaires, vous devez également faire la trace des trois premières valeurs de $playerRow$ et $playerCol$, ainsi que de k .

Cette trace sera à faire dans les cas suivants :

1. $playerRow = \{1,2,-1\}$; $playerCol = \{2, 2, -1\}$; $incR = 0$; $incC = 1$; $tokenRow = 1$; $tokenCol = 3$
2. $playerRow = \{3,4,-1\}$; $playerCol = \{1, 1, -1\}$; $incR = -1$; $incC = 0$; $tokenRow = 2$; $tokenCol = 3$

7 Fonctionnalités supplémentaires (optionnel)

Il est à noter que le développement d'un jeu n'est jamais vraiment terminé ! Après tout, qui vous empêche d'ajouter un score (par exemple 100 points par feuille mangée) ? Qui vous empêche de donner plusieurs vies à votre chenille ? Qui vous empêche d'ajouter des obstacles inoffensifs (ou pas) ? Ou bien d'autres choses !